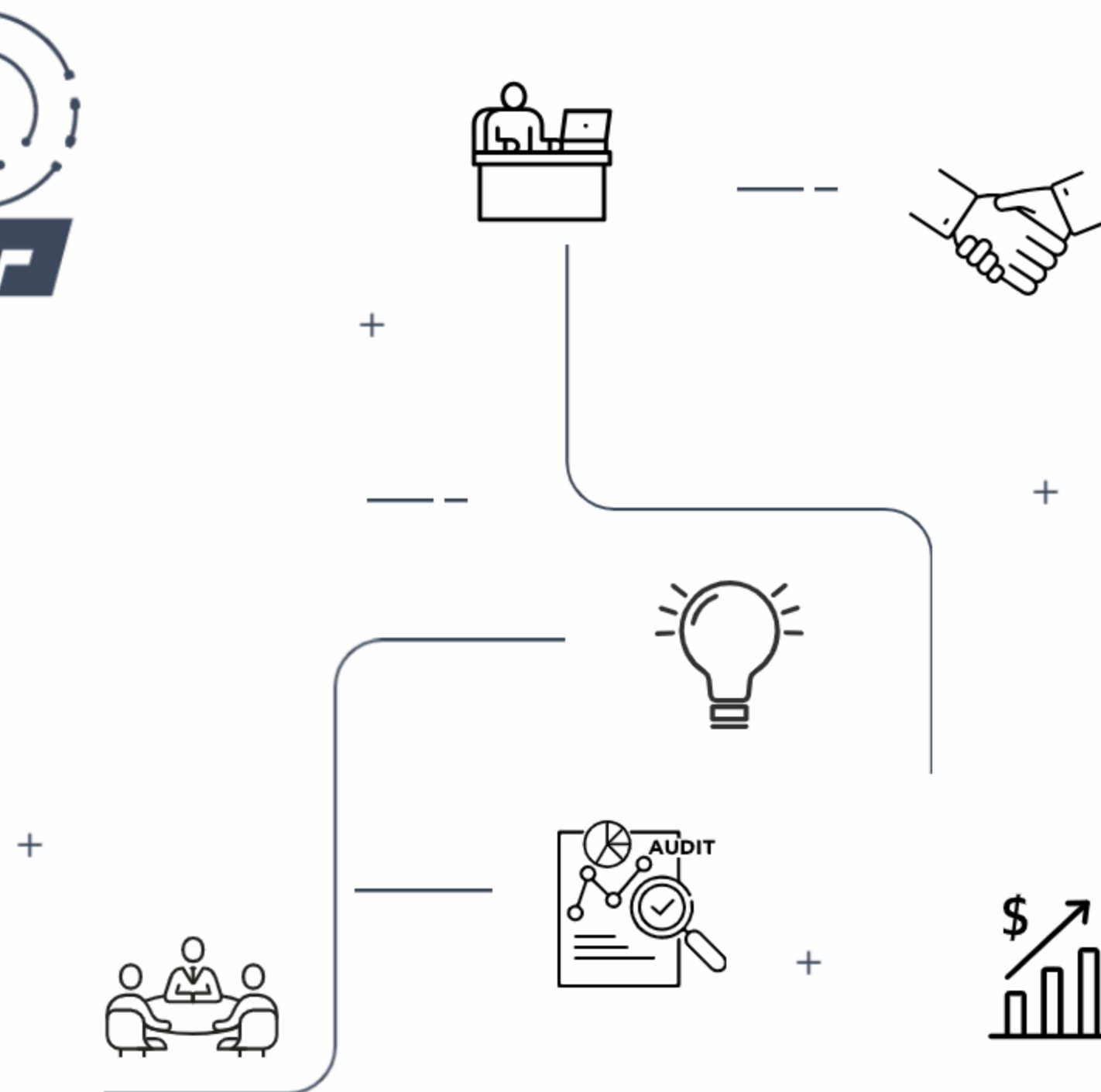




정우성 : <https://github.com/weare2415>

Github <https://github.com/paesir-i-am/NextERP>

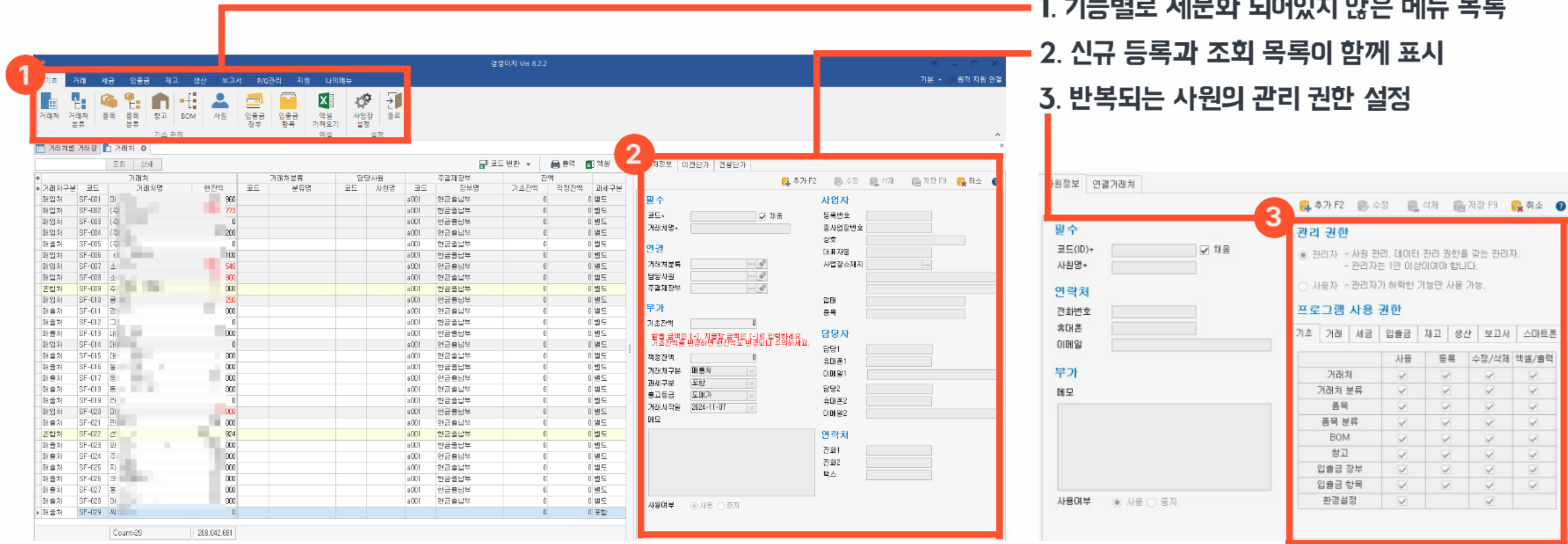
시연 영상 <https://www.youtube.com>



목차

1. 프로젝트 배경
2. 프로젝트 구현 기능(나의 역할 풀스택)
3. 핵심 기능 & 개발 도구
4. 수행 절차
5. 기능 설명
6. 문제 해결
7. 느낀점

1. 프로젝트 배경



1. 프로젝트 배경

1

HOME

공지사항

마이페이지

회계

영업

거래처 관리

제품 관리

주문 내역 조회

인사

운영관리

HOME

공지사항

마이페이지

회계

영업

인사

직원 관리

출/퇴근 관리

근태 신청 내역

운영관리

NEXT.ERP

황수림 이사님

영업팀

HOME

공지사항

마이페이지

회계

급여 관리

영업

인사

운영관리

1. 기능별로 간편하게 분리된 메뉴바

2. 조회 목록과 신규 등록을 분리하여 심플하게 구현

3. 팀과 직급에 따른 자동 권한 설정

2

급여 관리

직원 이름 검색

검색

+ 급여 추가

사원번호	직원명	직급	연봉	월급	적용 시작일
80001234	관리자	사장	50,000,000원	3,750,000원	2025-02-25
10000001	영업팀 사원	사원	32,000,000원	2,416,667원	2025-02-25
10000002	영업팀 대리	대리	35,000,000원	2,625,000원	2025-02-25
10000003	영업팀 부장	부장	42,000,000원	3,166,667원	2025-02-25
20000001	회계팀 인턴	인턴	28,000,000원	2,125,000원	2025-02-25
20000002	회계팀 과장	과장	39,000,000원	2,933,333원	2025-02-25
30000003	인사팀 부장	부장	43,000,000원	3,233,333원	2025-02-25
10001234	황수림	이사	55,000,000원	4,166,667원	2025-02-25
10007777	박남수	이사	60,000,000원	4,541,667원	2025-02-25
20001234	이정현	이사	48,000,000원	3,625,000원	2025-02-25

<< 1 2 >>

3

신규 직원 등록

사원 번호 (8자리 숫자)	70000808	이름	공서연
생년월일	1998-08-13	성별	여자
전화번호	010-4342-6357	이메일	aaa@naver.com
주소	서울 종로구	부서	영업팀
직급	사원	입사일	2025-02-27

등록

1. 프로젝트 배경

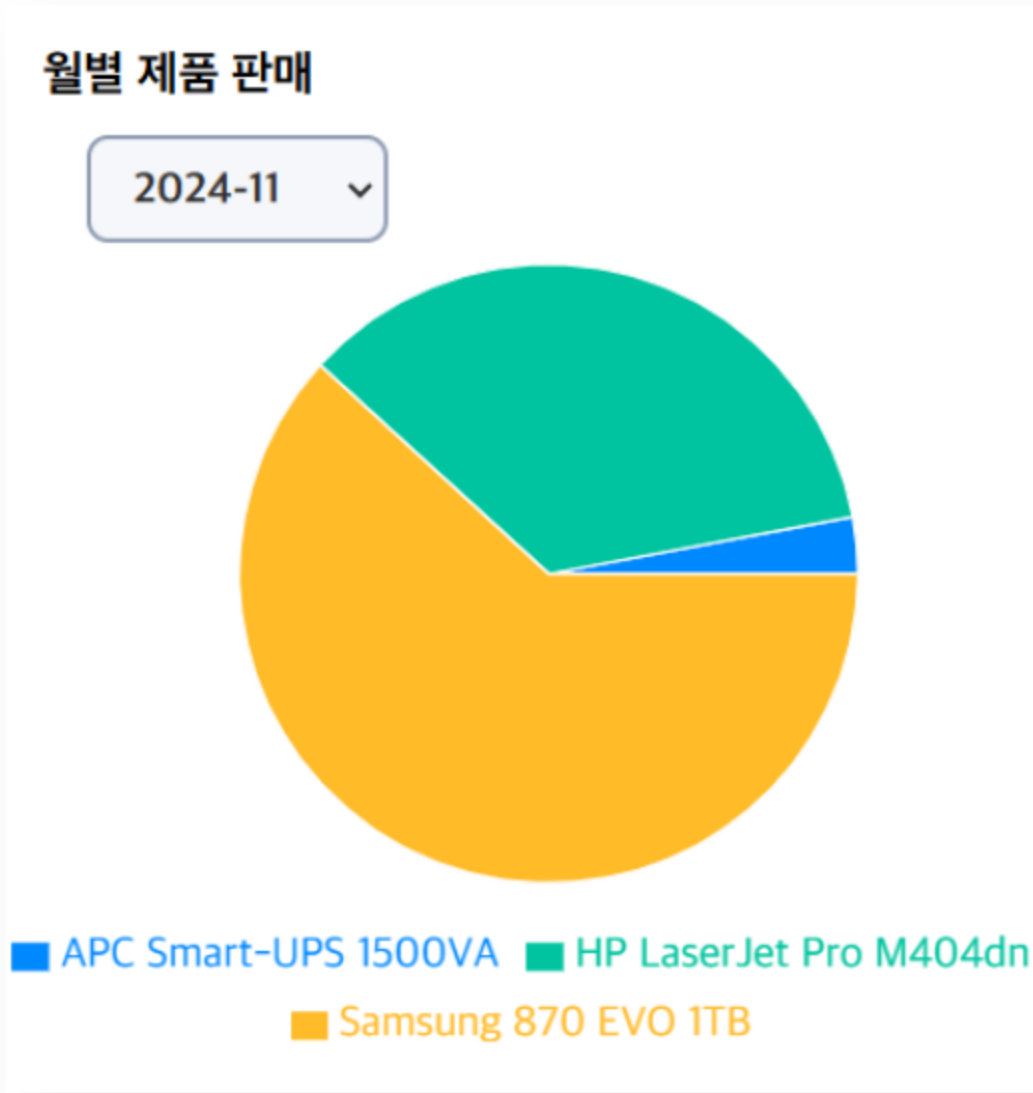
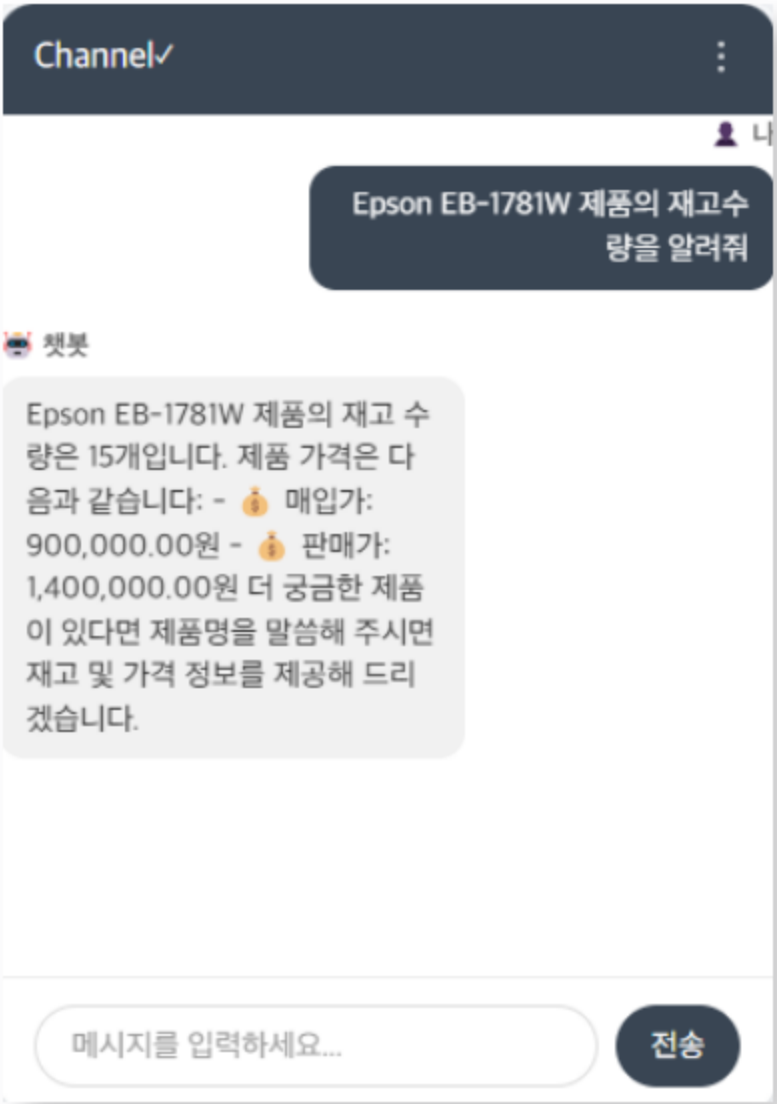
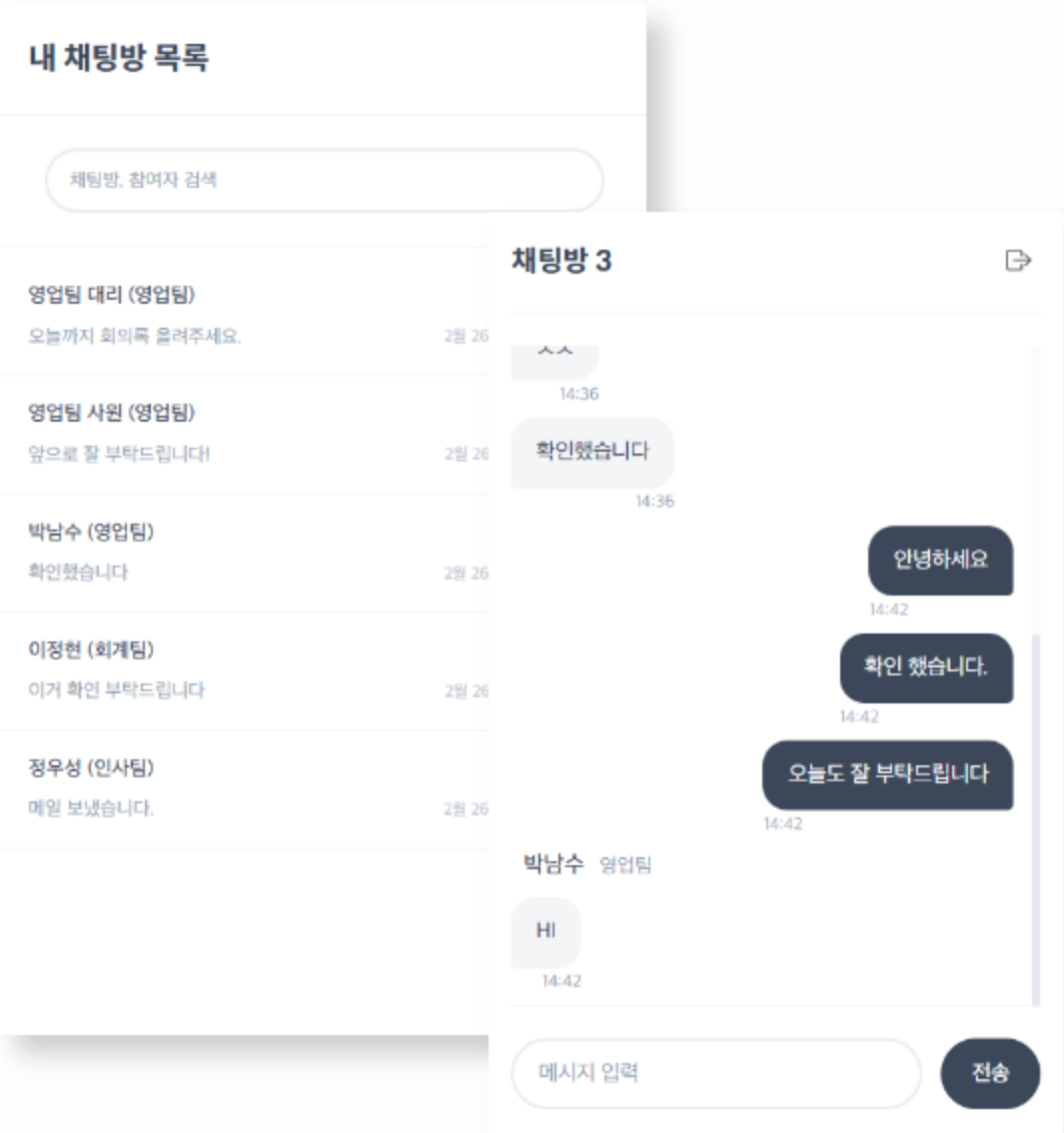
NEXT ERP는 실시간 **사내 메신저**, 24시간 언제든지 사용 가능한 **AI 챗봇**, 기존의 데이터를 기반으로 월별 영업 실적 **그래프**를 반영하여 즉각적인 **데이터 조회** 및 사용자의 **업무 효율성을 향상** 시켰습니다.

또한, 회사 내뿐만 아니라 언제 어디서든 접근이 가능한 서버를 구현하여, **ERP 시스템의 접근성과 편리함을 개선**하였습니다.

실시간
사내 메신저

업무 효율 상승
AI 챗봇

월별 영업 실적
그래프



2. 프로젝트 구현 기능(나의 역할 풀스택)

구현 기능	세부 내용
챗봇	Python을 사용해서 OpenAI 및 LangChain을 사용하여 자연어 처리를 수행하고, 사용자의 질문에 맞는 정보를 제공하도록 구현 각 부서, 직급별 권한을 고려 하여 맞춤형 답변 및 OpenAI 프롬프트를 통해 자연스럽게 응답 처리 <u>MariaDB 데이터를 조회</u> 하여 근태, 공지사항, 직원 정보, 거래내역을 실시간으로 확인
사내 메신저	Python과 Socket, Restful을 사용하여 <u>실시간으로 메시지 전달</u> 가능 생성된 채팅방과 그 채팅방에 나눈 대화는 DB에 저장되며 채팅방을 나갔다가 다시 생성할 때 이전 기록이 남아있기 때문에 이전 채팅 내역 확인 가능 읽지 않은 메시지의 경우 베이직 레이아웃에 있는 '메신저'옆, 채팅 목록에서 실시간으로 확인 가능
SMTP	비밀번호를 잊어버린 사용자가 비밀번호를 찾기 위해 1차적으로 사원ID를 입력하여 회사의 사원임을 인증 이후 이름과 이메일을 입력해서 2차 검증을 받은 이후 사용자가 입력한 메일주소로 <u>임시 비밀번호 전송</u> 전송 된 임시 비밀번호를 입력하면 로그인 가능
CRUD	인사팀(공지사항, 근태관리, 직원관리) 기본적인 CURD 구현 스케줄러 이용한 회사 자동처리 구현 단 수정을 할 때에는 권한 설정을 했기 때문에 바로 수정이 불가능하고 수정에 대한 승인 요청을 보내고 '승인' 처리가 돼야 실제 데이터가 수정됨 <u>페이지네이션</u> 을 컴포넌트화해서 여러 컴포넌트에서 손쉽게 페이지네이션 기능 적용
권한 처리	로그인 시 Redux의 로그인 슬라이스를 활용하여 사용자 정보를 조회하고, 해당 회원의 부서 ID 및 직급 ID를 식별하여 동적으로 권한을 부여하도록 구현 이를 통해 각 사용자가 소속된 부서 및 직급에 따라 <u>접근 가능한 메뉴 및 기능을 제어</u> 하며, <u>사용자별 맞춤형 시스템 환경</u> 을 제공함
API	Spring Mail, Redux Toolkit, Axios,

3. 핵심 기능 & 개발 도구

분야	개발도구	상세 설명
Back-End	Java 17 Spring Boot 3 Spring Security JWT	Spring Security 및 JWT를 활용하여 보안성 강화 JPA Repository를 이용하여 쿼리메서드를 사용하여 로직을 간결하고 직관적으로 구현 RestController를 사용해 RESTful 방식을 사용하여 프론트엔드와 원활한 통신
Front-End	React Node.js JavaScript Recharts Socket.IO HTML5 CSS3 SCSS	React를 사용하여 SPA기반으로 반응형 웹 페이지를 구현 Rouer를 사용하여 URL 파라미터의 관리가 용이 Reducer와 Redux를 통해 전역 상태 관리를 구현하여 데이터를 효율적으로 처리 Custom Hook을 사용하여 코드의 재사용 및 편의성 증가 Axios 와 AJAX를 사용하여 백엔드와의 비동기 통신으로 불필요한 통신을 줄이고 응답속도 향상 Recharts를 사용하여 직관적인 데이터 시각화를 통해 월별 영업 실적 그래프와 같은 시각적 데이터를 효율적으로 구현 Socket.IO를 사용하여 서버와 클라이언트 간 소켓 이벤트 처리 및 실시간 사내 메신저 구현
데이터	MariaDB Python3	MariaDB 사용하여 데이터베이스 효율적으로 생성 및 관리 Python을 사용하여 AI 챗봇과 실시간 사내 메신저 구현
Github	협업관리	Github 사용하여 프로젝트 협업관리

4. 수행 절차 - WBS

[illegible]

4. 수행 절차

구분	기간	활동	비고
기획	1/13(월)~1/14(화)	아이디어 브레인 스토밍 프로젝트 기획 및 주제 선정 기능 정리 및 사용할 라이브러리 조사	참고 사이트 이카운트, 다우오피스, 더존, 영림원
설계	1/15(수)~1/17(금)	로고 제작 및 Basiclayout 구현 팀별 역할 분담 & ERD 설계 팀별 뷰 페이지 구상도 작성	백단 모델링 설계 뷰 페이지 구상도
구현	1/17(금)~1/19(일) 2/10(월)~2/21(금)	팀별 백 & 프론트 구현 메인 페이지 월별 매출 그래프 구현 챗봇 & 실시간 사내 메신저 구현	챗봇 : Python 사내 메신저 : Socket.IO 그래프 : Recharts
최종구현	2/22(토)~2/27(목)	구현한 페이지 병합 및 에러 처리 반응형 CSS 수정	페이지 합기 서류 작업 : 2/25~2/27

권한 설정

직급 선택

주소

주소

부서

부서 선택

▼

직급

직급 선택

▼

입사일

연도-월-일

📅

직급 선택

인턴

사원

등록

부서, 지금 권한 부여

```
// 가져온 사용자 정보에서 departmentId, positionId 가져오기
const departmentId = user.departmentId ?? 0;
const positionId = user.positionId ?? 0;

console.log("👤 현재 유효한 departmentId:", departmentId);
console.log("👤 현재 유효한 positionId:", positionId);

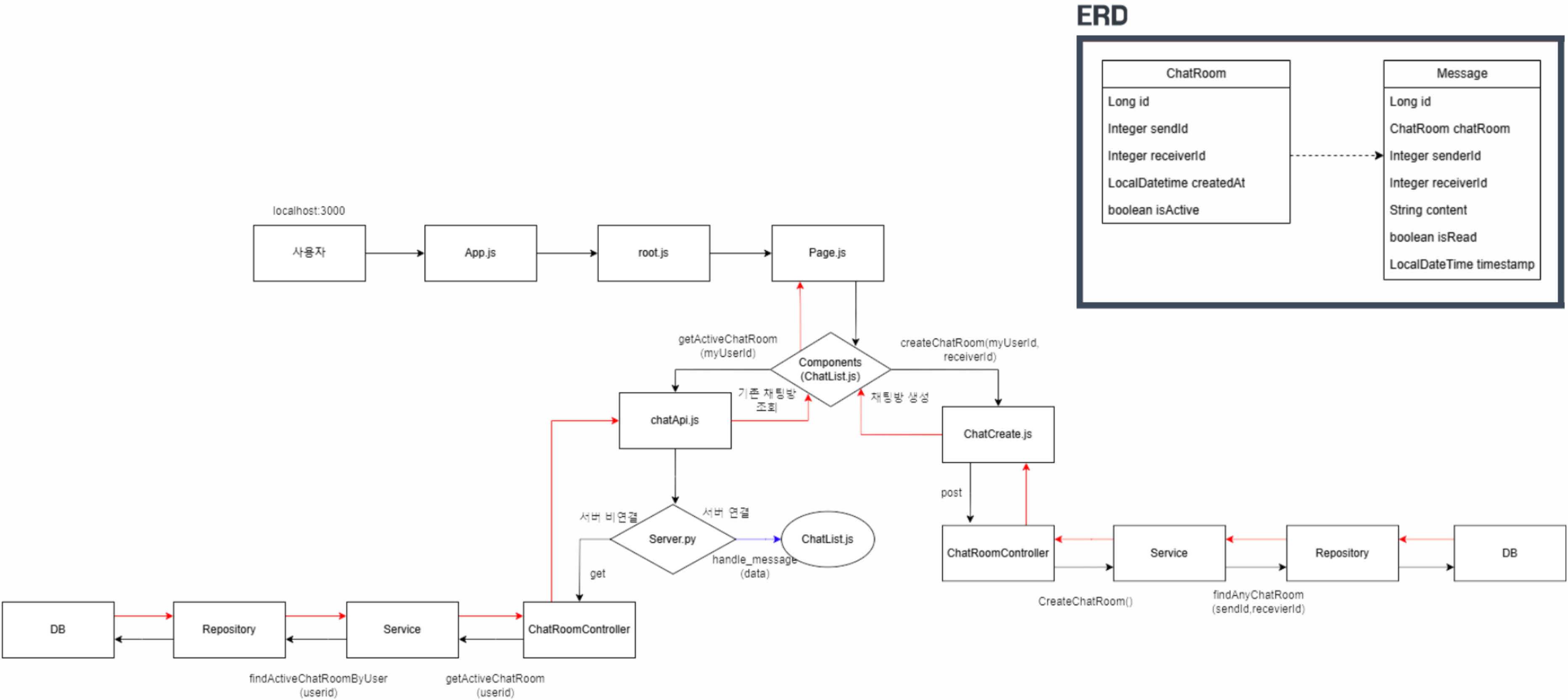
// 이사(7), 사장(8)은 모든 메뉴 접근 가능
const isSupervisor = positionId >= 7;

const filteredMenuItems = {
  HOME: [{ name: "HOME", path: "/" }],
  공지사항: [{ name: "공지 사항", path: "/announcement" }],
  마이페이지: [
    { name: "My 출/퇴근 조회", path: "/employee/myattendance" },
    { name: "근태 신청", path: "/employee/attendance/request" },
  ],
  회계:
    isSupervisor || departmentId === 2
      ? [{ name: "급여 관리", path: "/payroll" }]
      : [],
  영업:
    isSupervisor || departmentId === 1
      ? [
          { name: "거래처 관리", path: "/clients" },
          { name: "제품 관리", path: "/product" },
          { name: "주문 내역 조회", path: "/product/order" },
        ]
      : [],
};
```

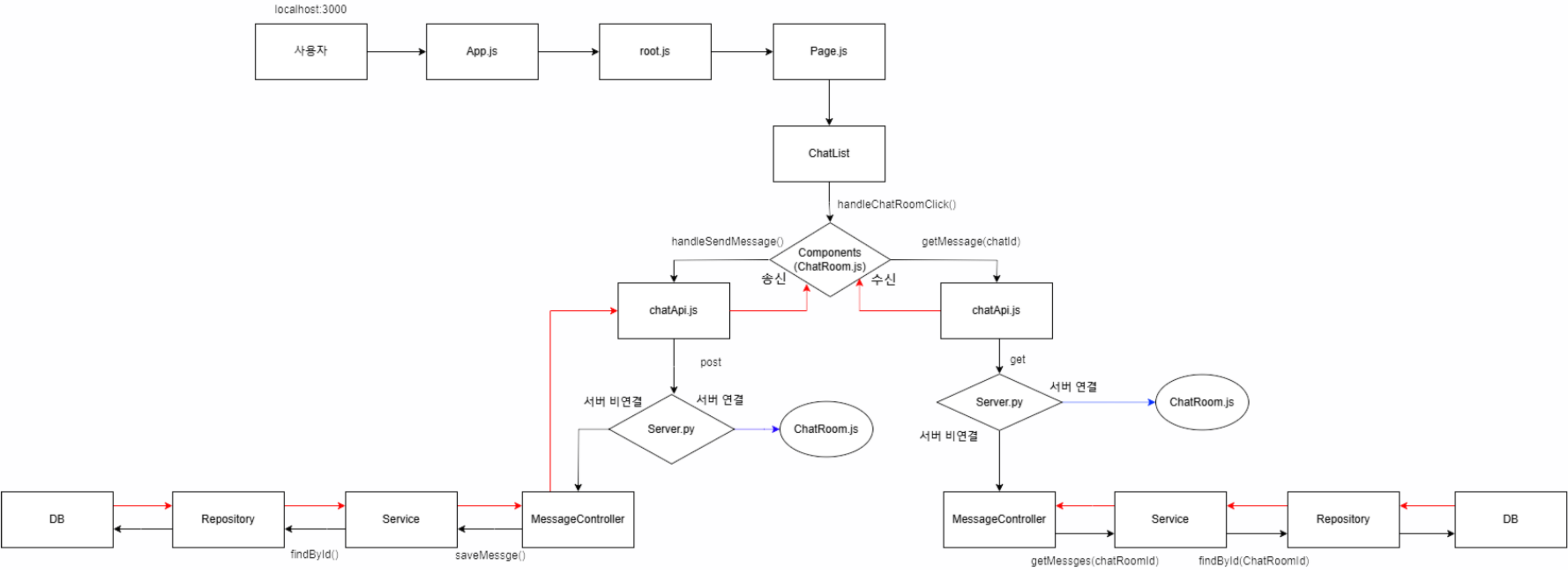
권한별 접근 권한 항목

권한	사원	팀장	이사	사장
공지사항	0	0	0	0
마이페이지	0	0	0	0
회계	회계팀만 접근 가능하고 이사급 이상만 접근 가능			
거래처, 제품, 주문 조회 (영업팀)	영업팀만 접근 가능하고 이사급 이상만 접근 가능			
직원 관리, 근태 신청 및 조회 (인사팀)	인사팀만 접근 가능하고 이사급 이상만 접근 가능			
운영관리(영업팀) 거래처 수정 승인, 주문 승인	X	0	0	0
운영관리(인사팀) 근태 승인, 사원 수정 승인	X	0	0	0

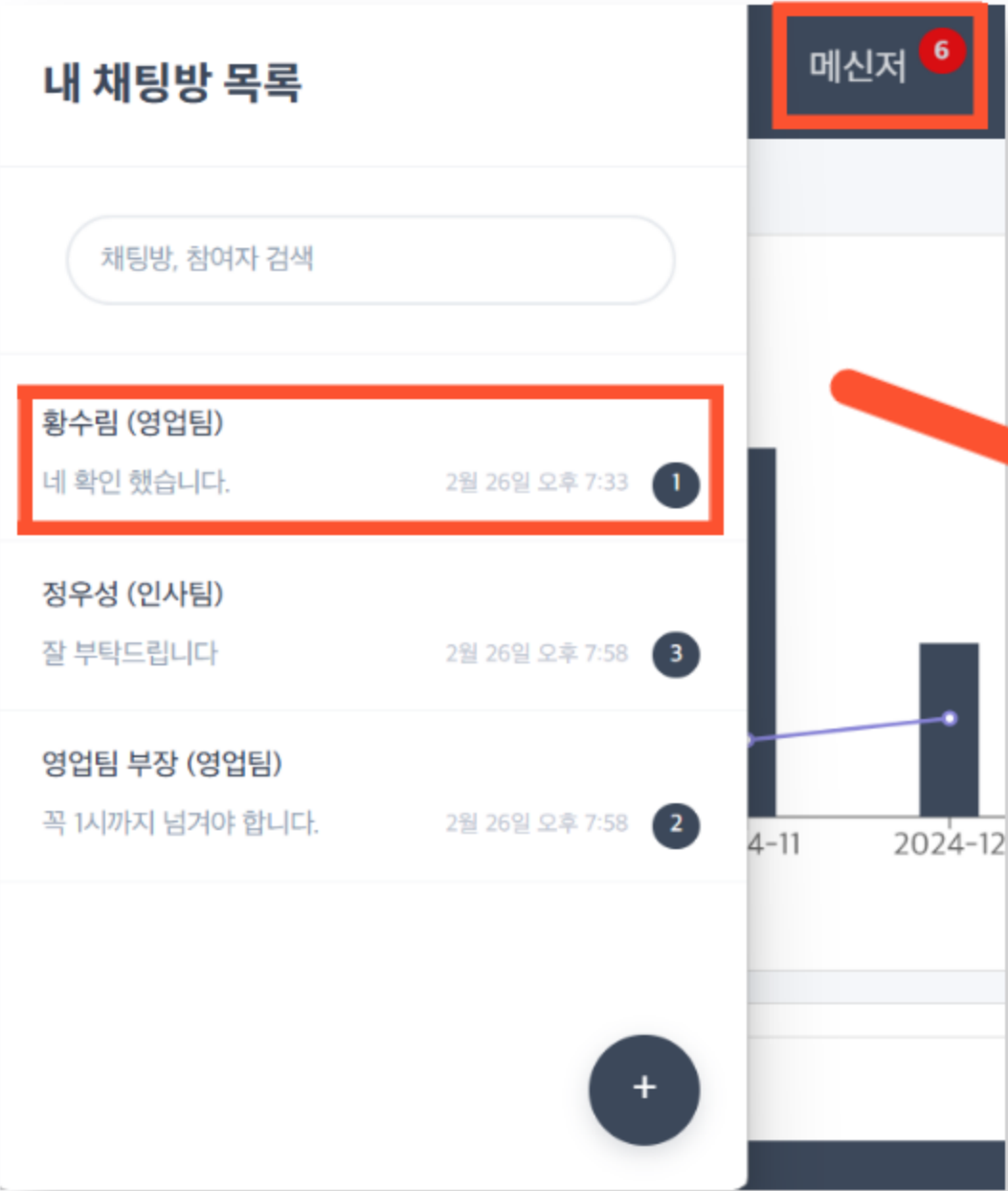
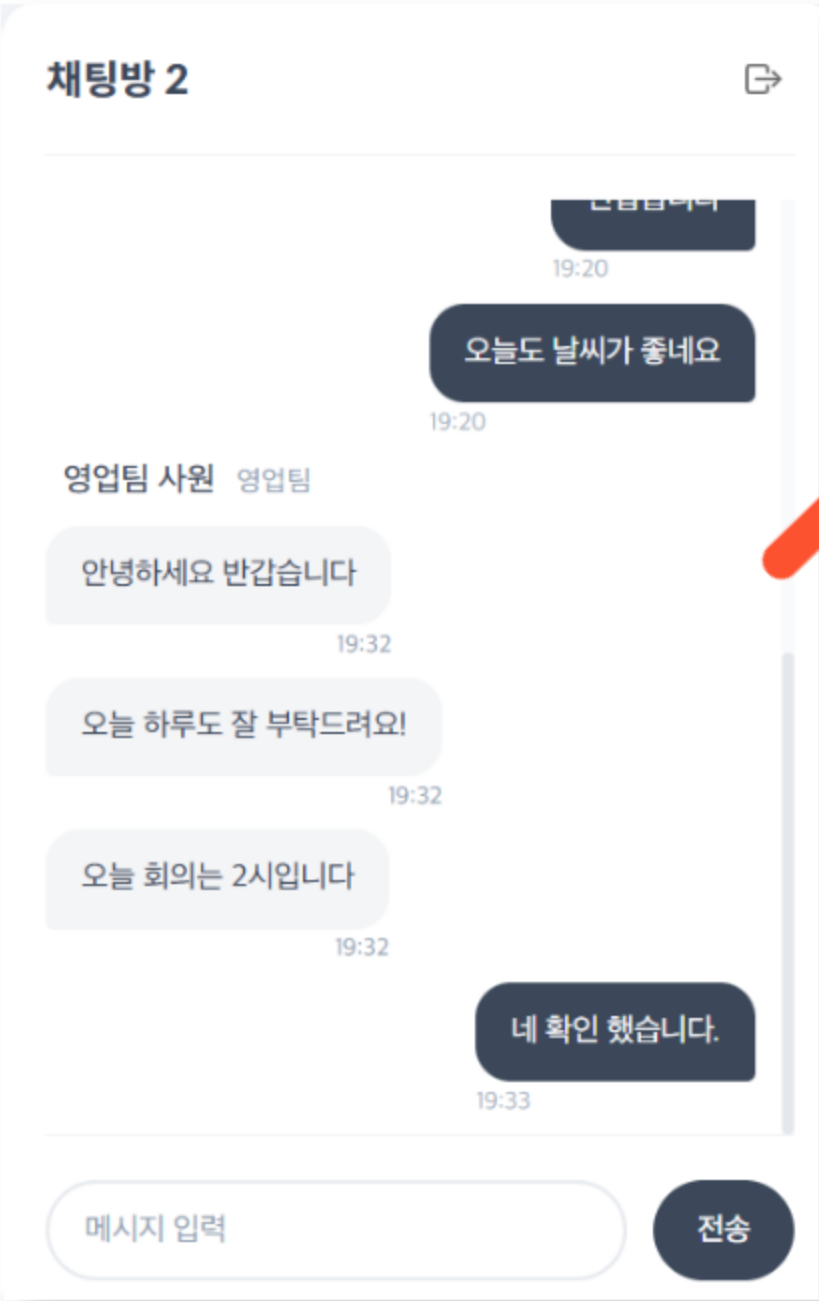
사내 메신저 - 조회, 생성 플로우 차트 & ERD



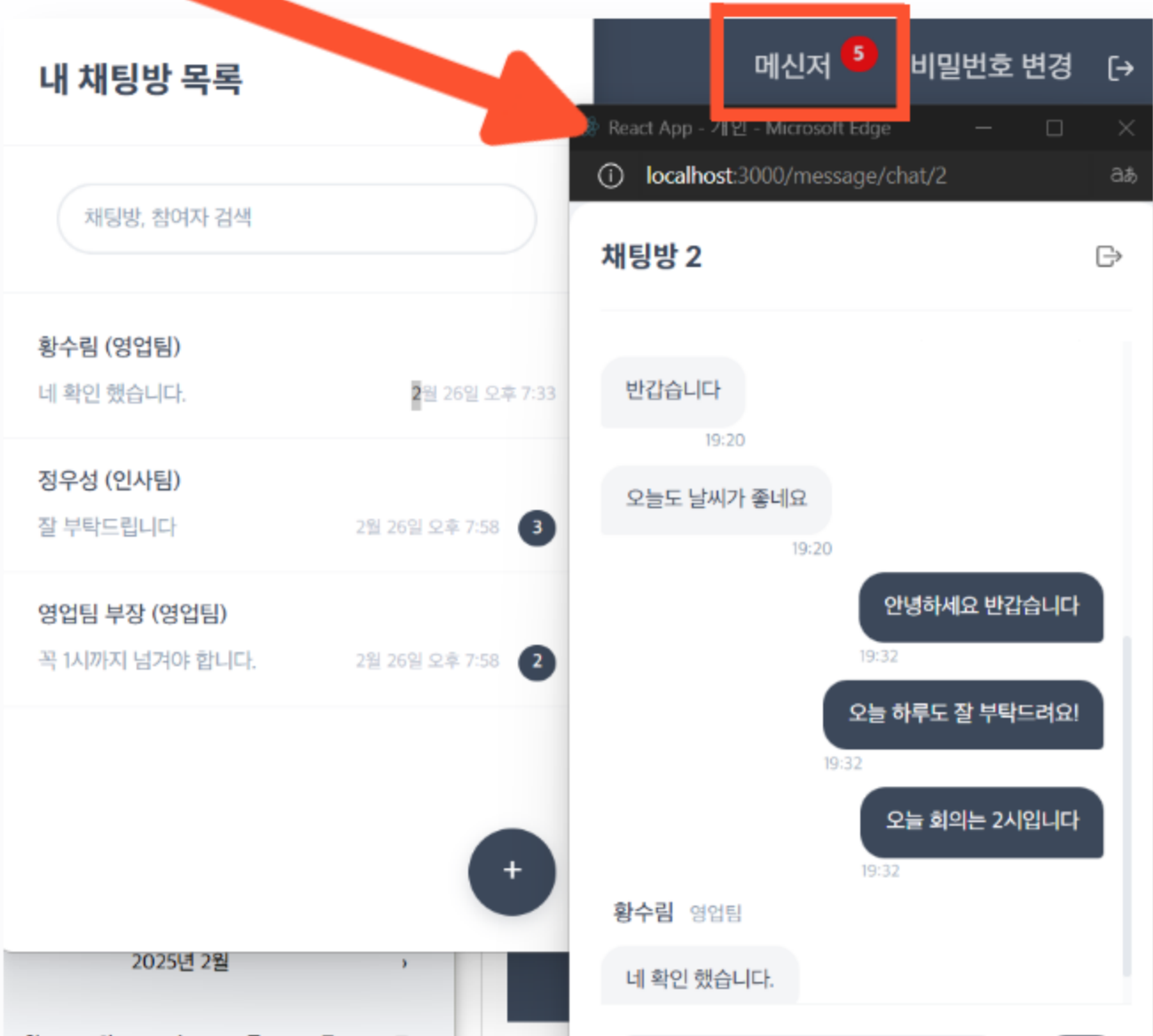
사내 메신저 - 전송, 수신 플로우 차트



사내 메신저



1. 읽지 않은 메시지는 Socket 이벤트를 사용하여 실시간으로 베이직 레이아웃의 '메신저' 옆에 총 합산되어서 표시
2. 메신저는 실시간으로 전송 및 수신 가능



사내 메신저



FE 메시지 전송할 때 동작하는 메서드,

```
// 메시지 전송 함수
const handleSendMessage = async () => {
  if (!message.trim()) return;

  const currentTimeStamp = new Date().toISOString();
  const formattedTimestamp = formatTime(currentTimeStamp);

  const messageData = {
    chatRoomId,
    senderId: myUserId,
    receiverId,
    messageText: message.trim(),
    content: message.trim(),
    timestamp: currentTimeStamp,
    jwtToken,
  };

  console.log("📬 메시지 전송:", messageData);

  socket.emit("message", messageData);

  const savedMessage = await sendMessage(
    chatRoomId,
    myUserId,
    receiverId,
    message.trim()
  );
};
```



Socket.IO 이벤트 발생

```
@socketio.on("message")
def handle_message(data):
    chat_room_id = data.get("chatRoomId")
    sender_id = data.get("senderId")
    receiver_id = data.get("receiverId")
    message_text = data.get("messageText")

    print(f"📬 [Flask] 수신 데이터: {json.dumps(data, indent=2, ensure_ascii=False)}")

    if not message_text:
        print("❌ 메시지가 비어 있습니다.")
        return

    # WebSocket으로 보낼 메시지
    websocket_message = {
        "chatRoomId": chat_room_id,
        "senderId": sender_id,
        "receiverId": receiver_id,
        "messageText": message_text,
        "content": message_text, # React에서 사용
        "timestamp": datetime.datetime.now().isoformat(), # ✅ 현재 시간 추가
    }
```



BE 컨트롤러

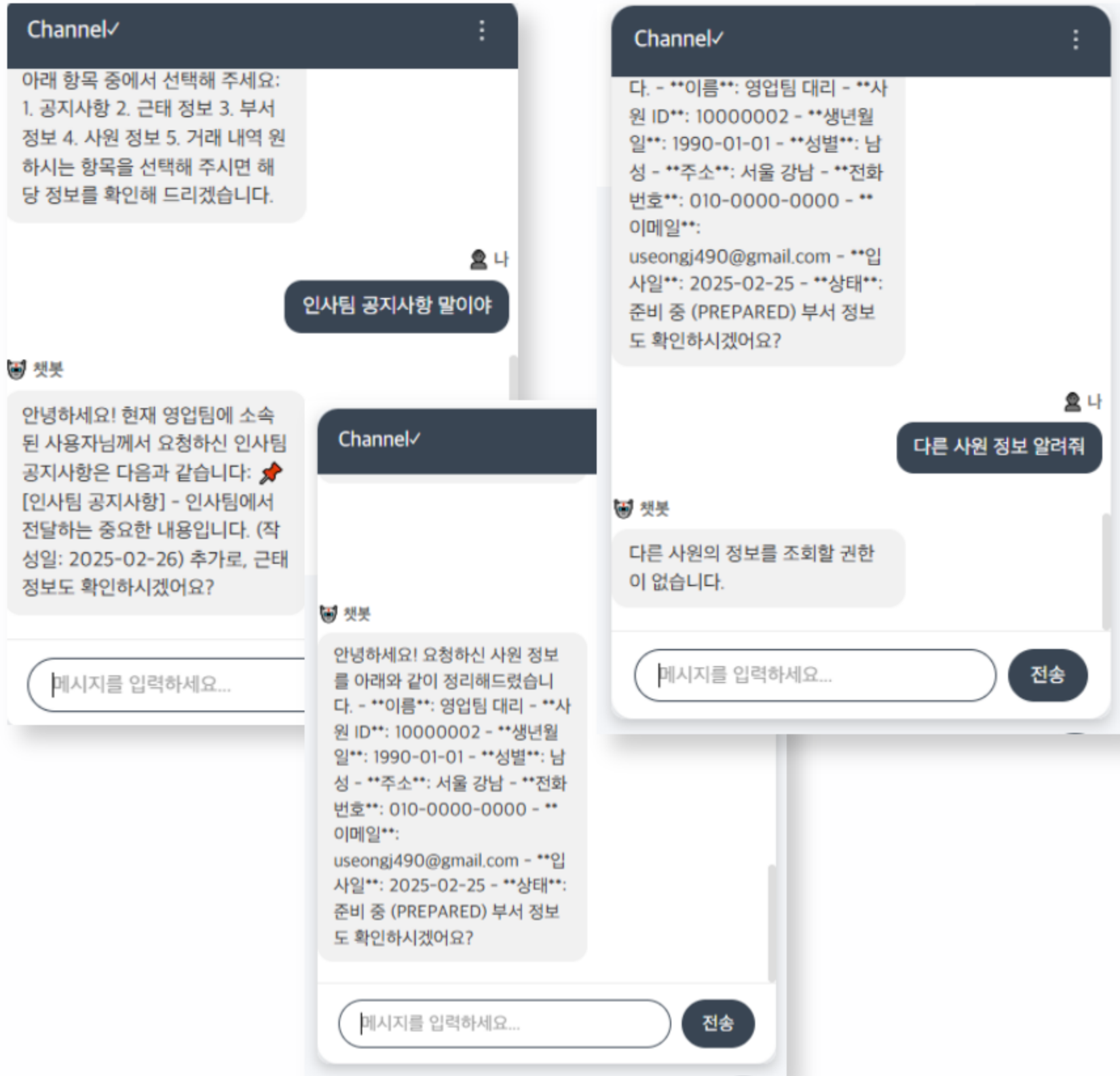
```
@PostMapping({"/send", "/"}) // ✅ "/messages"와 "/messages/send" 둘 다 허용 👤 qkrskatn01
public MessageResponseDTO sendMessage(@RequestBody MessageRequestDTO messageRequestDTO) throws JsonProcessingException {
    System.out.println("📬 [Spring Boot] 받은 메시지 요청: " + messageRequestDTO);

    return messageService.saveMessage(messageRequestDTO);
}
```

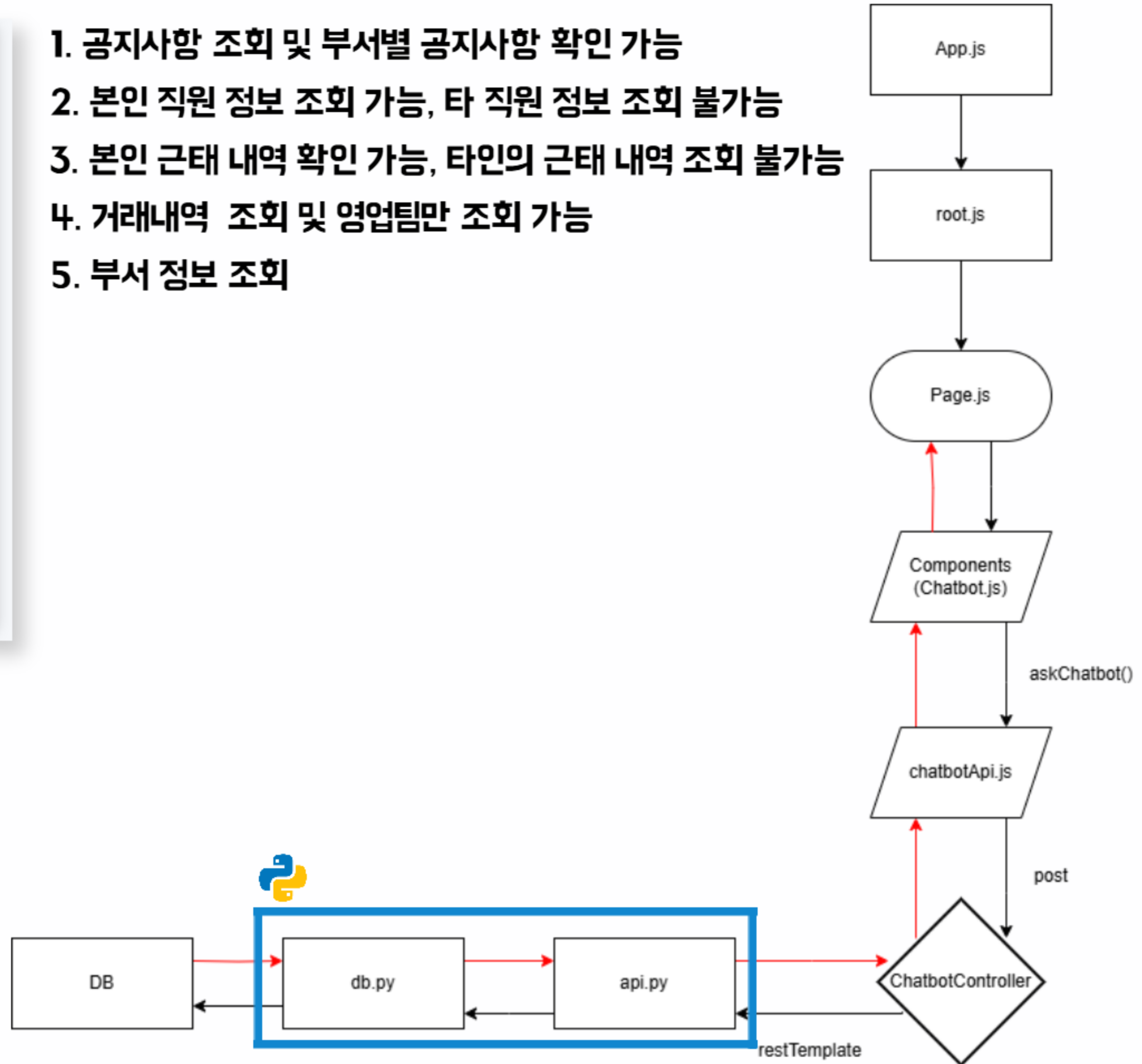
DB 데이터 삽입

WHERE		ORDER BY					
id	content	is_read	receiver_id	sender_id	timestamp	chat_room_id	
1	[{"id":null,"chatRoomId":1,"senderId":10000002,"receiverI...	true	10001234	10000002	2025-02-26 14:34:21	1	
2	[{"id":null,"chatRoomId":2,"senderId":10000001,"receiverI...	true	10000001	10001234	2025-02-26 14:34:56	2	
3	[{"id":null,"chatRoomId":3,"senderId":10007777,"receiverI...	false	10007777	10001234	2025-02-26 14:35:44	3	
4	[{"id":null,"chatRoomId":4,"senderId":20001234,"receiverI...	true	10001234	20001234	2025-02-26 14:37:06	4	
5	[{"id":null,"chatRoomId":5,"senderId":30001234,"receiverI...	true	10001234	30001234	2025-02-26 14:37:55	5	
6	[{"id":null,"chatRoomId":6,"senderId":30001234,"receiverI...	false	10000001	30001234	2025-02-26 19:58:08	6	
7	[{"id":null,"chatRoomId":7,"senderId":10000003,"receiverI...	false	10000001	10000003	2025-02-26 19:58:49	7	

챗봇



1. 공지사항 조회 및 부서별 공지사항 확인 가능
2. 본인 직원 정보 조회 가능, 타 직원 정보 조회 불가능
3. 본인 근태 내역 확인 가능, 타인의 근태 내역 조회 불가능
4. 거래내역 조회 및 영업팀만 조회 가능
5. 부서 정보 조회



챗봇



```
# 데이터베이스 연결 정보
DB_CONFIG = {
    "host": "localhost",
    "port": 3306,
    "user": "paesir",
    "password": "386500",
    "database": "next-erp",
    "local_infile": True
}
```

DB 조회

```
def show_data(table):
    """ 특정 테이블의 데이터를 가져와 Pandas DataFrame으로 변환 """
    conn = None
    try:
        conn = mariadb.connect(**DB_CONFIG)
        cur = conn.cursor()

        # 데이터 조회
        cur.execute(f"SELECT * FROM {table}")
        columns = [x[0] for x in cur.description]
```

LangChain 활용 자연어 처리

```
# OpenAI 프롬프트 생성 (LangChain 활용)
if "사원" in user_query or "내 정보" in user_query:
    prompt = f"""
    너는 기업 내부 시스템의 챗봇이야. 사용자가 자신의 사원 정보를 요청했어.

    📌 사원 정보:
    {employee_info}

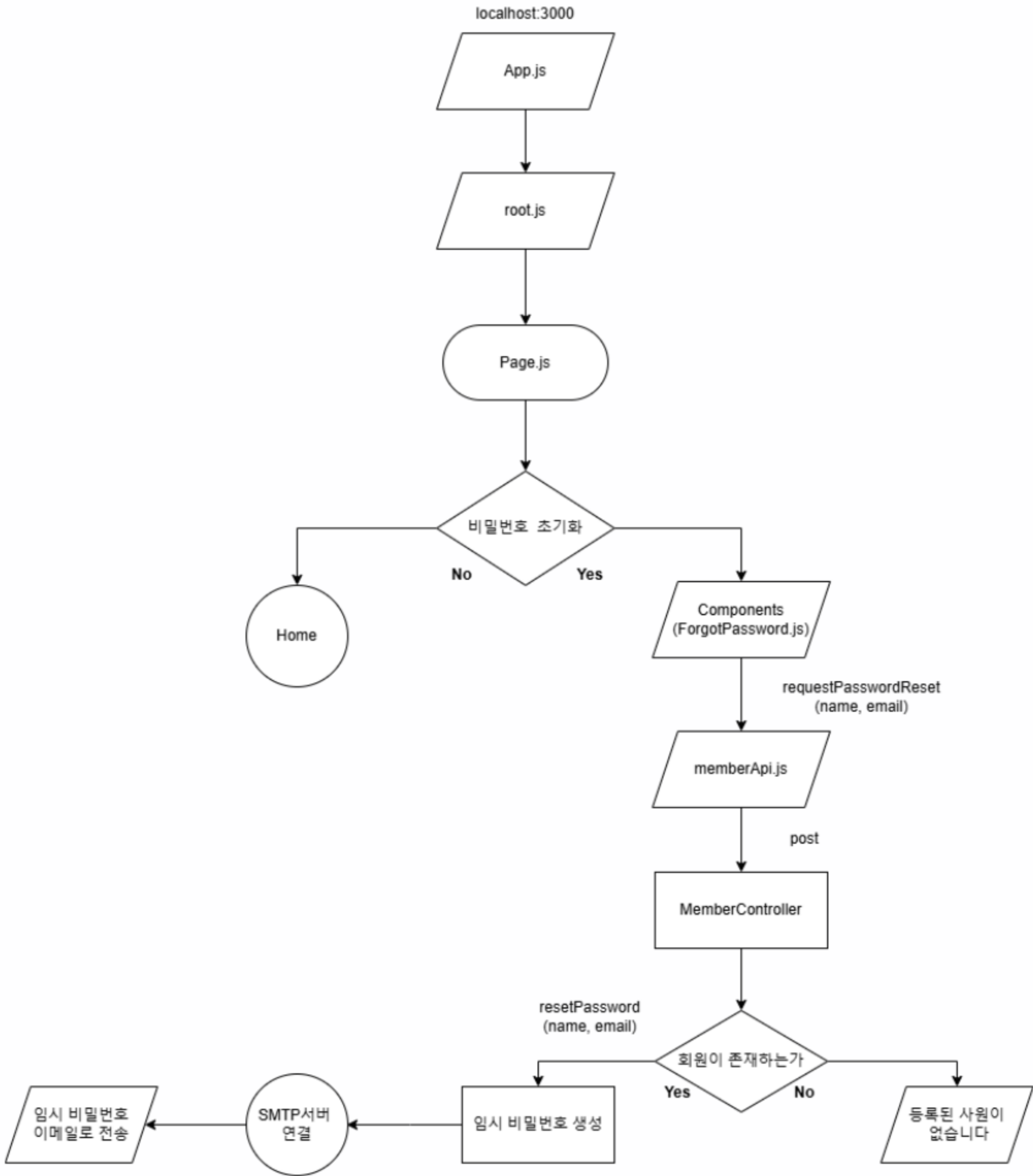
    사용자가 '{user_query}'라고 입력했어.
    그리고 다른 사원의 정보는 보여주면 안 될 거 같아
    자연스럽게 사원 정보를 정리해서 전달하고, 추가로 '부서 정보도 확인하시겠어요?' 라고 물어봐.
    """
elif "부서" in user_query or "부서 정보" in user_query:
    department_df = db.show_data("department")
    if department_df.empty:
        department_info = "❗ 현재 확인할 수 있는 부서 정보가 없습니다."
    else:
        department_list = [
            f"📌 {row['department_id']}번 부서: {row['name']} (연락처: {row['contact_email']})"
            for _, row in department_df.iterrows()
        ]
        department_info = "\n".join(department_list)
```

현재 로그인한 사용자만 조회

```
# 사원 정보 조회 (현재 로그인한 사용자만 조회)
employee_df = db.show_data("employee")

if employee_df.empty:
    employee_info = "❗ 현재 확인할 수 있는 사원 정보가 없습니다."
else:
    # 요청한 사원의 ID가 로그인한 사용자 ID와 다르면 조회 불가
    if "사원" in user_query and str(employee_id) not in user_query:
        return jsonify({"response": "다른 사원의 정보를 조회할 권한이 없습니다."})
    personal_employee = employee_df[employee_df["employee_id"] == employee_id]
    if personal_employee.empty:
        employee_info = "❗ 본인의 사원 정보를 찾을 수 없습니다."
    else:
        employee_info = personal_employee.to_dict(orient="records")[0] # JSON 형태로 변환
```


SMTP - 플로우 차트



SMTP 메일 발송



SMTP 초기 설정

```
spring.mail.host=smtp.gmail.com
spring.mail.port=465
spring.mail.username=useongj490@gmail.com
spring.mail.password=jnmjglrdhfgdxbjy
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.ssl.enable=true
spring.mail.properties.mail.smtp.starttls.enable=false
spring.mail.properties.mail.smtp.connectiontimeout=5000
spring.mail.properties.mail.smtp.timeout=5000
spring.mail.properties.mail.smtp.writetimeout=5000
```



사원번호

비밀번호

[forgot password?](#)

로그인

1. 'forgot password' 클릭시
사원 인증 페이지로 이동
2. 사원ID를 통한 1차 검증



사원 인증

비밀번호 찾기를 위해 사원 ID를 입력하고 인증을 진행하세요.

사원 ID:

인증하기



사원 ID 1차 검증

```
// | 사원 ID 검증
const handleVerify = async (e) => {
  e.preventDefault();
  setError(""); // 기존 에러 초기화

  if (!employeeId) {
    setError("사원 ID를 입력하세요.");
    return;
  }

  try {
    const employeeData = await getEmployeeById(employeeId); // API 호출
    console.log("사원 인증 성공:", employeeData);

    // 성공 메시지 알람 표시 후 페이지 이동
    window.alert("인증이 완료되었습니다! 비밀번호 찾기 페이지로 이동합니다.");

    // ✅ 인증 성공 시 비밀번호 찾기 페이지로 이동 (사원 ID 전달)
    navigate("/member/forgot-password", { state: { employeeId } });
  } catch (error) {
    setError("❌ 사원 ID가 존재하지 않거나, 잘못된 ID입니다.");
  }
};
```

SMTP 메일 발송



비밀번호 찾기

등록된 직원의 이름과 이메일을 입력하면 임시 비밀번호가 발송됩니다.

이름:

이메일:

임시 비밀번호 요청

비밀번호 초기화 안내

우성

useongj490@gmail.com

나에게 ▼

안녕하세요, 영업팀 직원님.

비밀번호가 초기화되었습니다.

새 비밀번호: **PVMiHMKv**

로그인 후 반드시 변경해 주세요.

요청을 받은 후 2차 검증

```
@PostMapping("/forgot-password")
public ResponseEntity<?> forgotPassword(@RequestBody ForgotPasswordRequestDTO request) {
    try {
        System.out.println("요청 데이터: " + request);

        String name = request.getName();
        String email = request.getEmail();

        if (name == null || email == null || name.isEmpty() || email.isEmpty()) {
            return ResponseEntity.badRequest().body(Map.of("message", "이름과 이메일을 입력하세요."));
        }

        memberService.resetPassword(name, email);
        return ResponseEntity.ok().body(Map.of("message", "임시 비밀번호가 이메일로 전송되었습니다."));
    } catch (IllegalArgumentException e) {
        System.out.println("예외 발생: " + e.getMessage());
        return ResponseEntity.badRequest().body(Map.of("message", e.getMessage()));
    } catch (Exception e) {
        System.out.println("서버 오류: " + e.getMessage());
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(Map.of("message", "서버에서"));
    }
}
```

비밀번호 초기화 및 이름과 이메일로 ID 조회 2차 검증

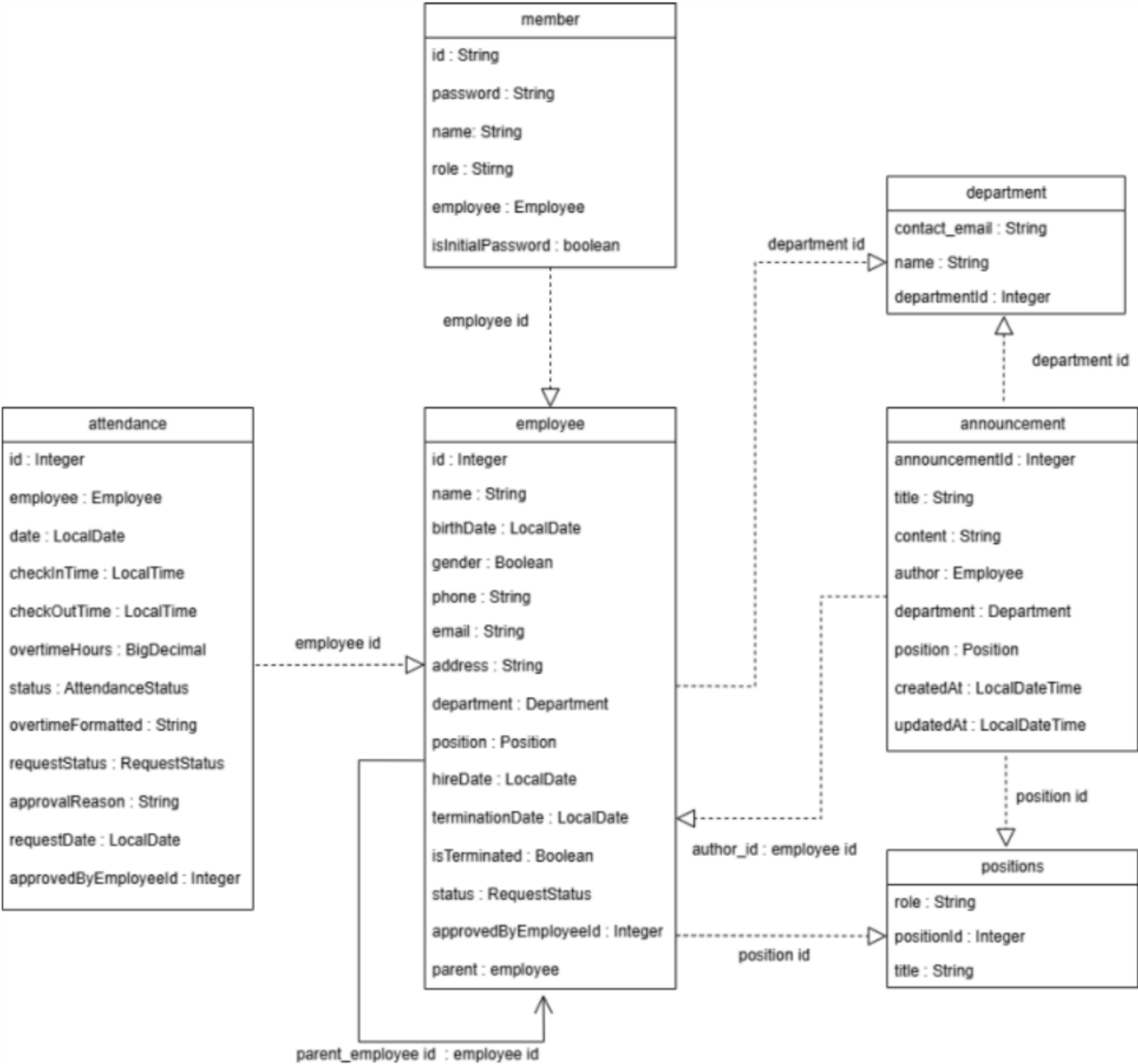
```
// **비밀번호 초기화**
public void resetPassword(String name, String email) {
    // 1. 이름과 이메일로 Employee 조회 (id 찾기)
    Employee employee = employeeRepository.findByNameAndEmail(name, email)
        .orElseThrow(() -> new IllegalArgumentException("이름 또는 이메일이 일치하는 회원이 없습니다."));
    logger.info("Employee 정보: {}", employee);
}
```

해당 이메일로 전송

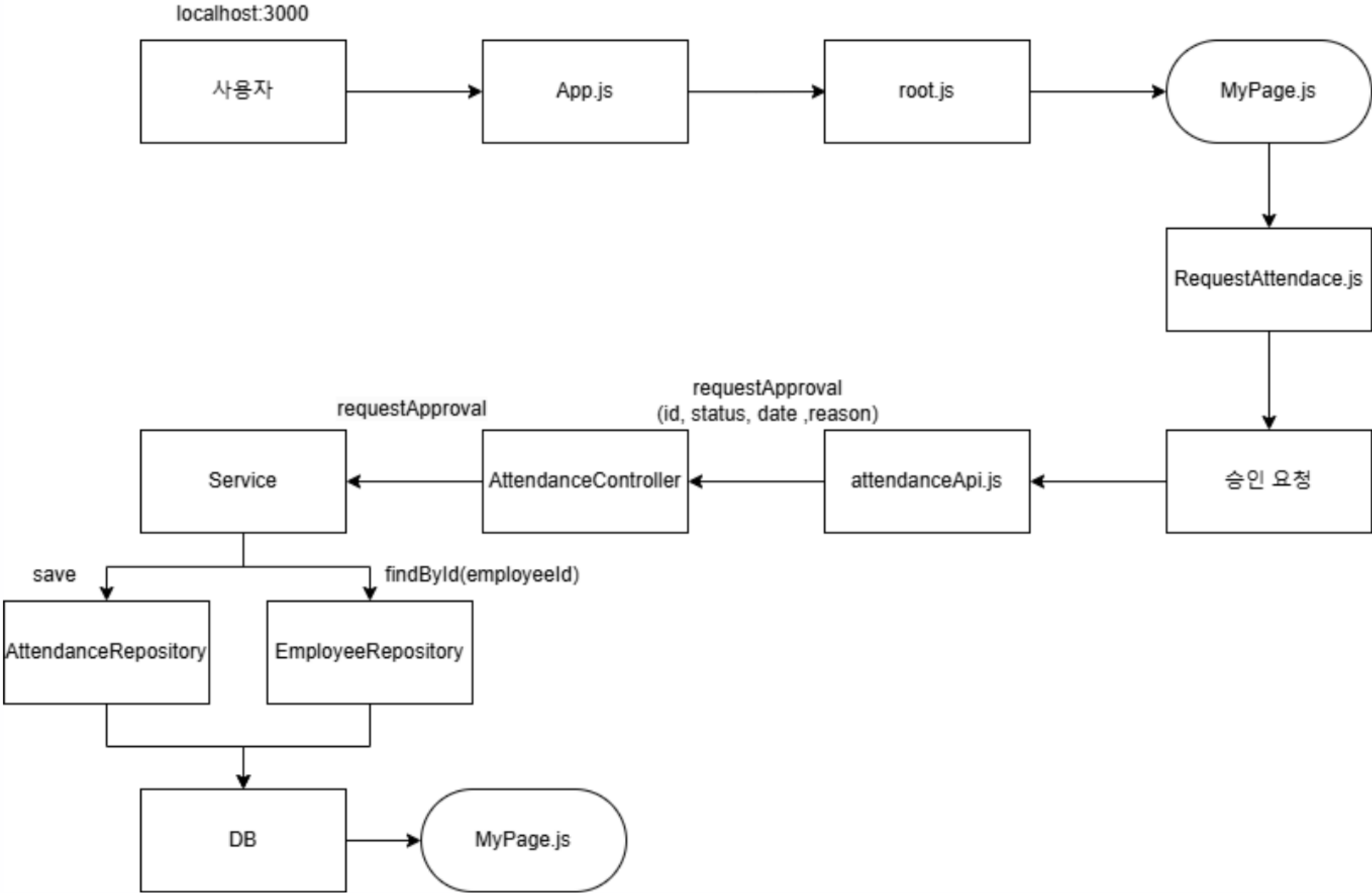
```
private void sendPasswordResetEmail(String recipientEmail, String recipientName, String newPassword) {
    try {
        // 수신자 이메일 로그 확인
        logger.info("이메일 전송 시도 - 받는 사람: [{}]", recipientEmail);

        MimeMessage message = mailSender.createMimeMessage();
        MimeMessageHelper helper = new MimeMessageHelper(message, multipart: true, encoding: "UTF-8");
    }
}
```

인사팀(공지사항, 직원관리, 근태관리) - ERD



근태 신청 - 플로우 차트



근태 신청 및 내역

근태 신청

휴가 & 병가 내역

휴가 쓴 일수: 0 / 12

날짜	근태 유형	사유	승인 상태
2025-03-29	휴가		반려
2025-03-28	휴가	휴가	승인 대기
2025-03-19	병가	병가	승인 대기

<< 1 >>

근태 신청 X

근태 유형

휴가

신청 날짜

연도-월-일

신청 사유

사유를 입력하세요.

신청하기

```
@Transactional
public AttendanceDTO requestApproval(Integer employeeId, String status, LocalDate date, String reason) {
    AttendanceStatus newStatus = AttendanceStatus.valueOf(status.toUpperCase());
    LocalDate today = LocalDate.now();

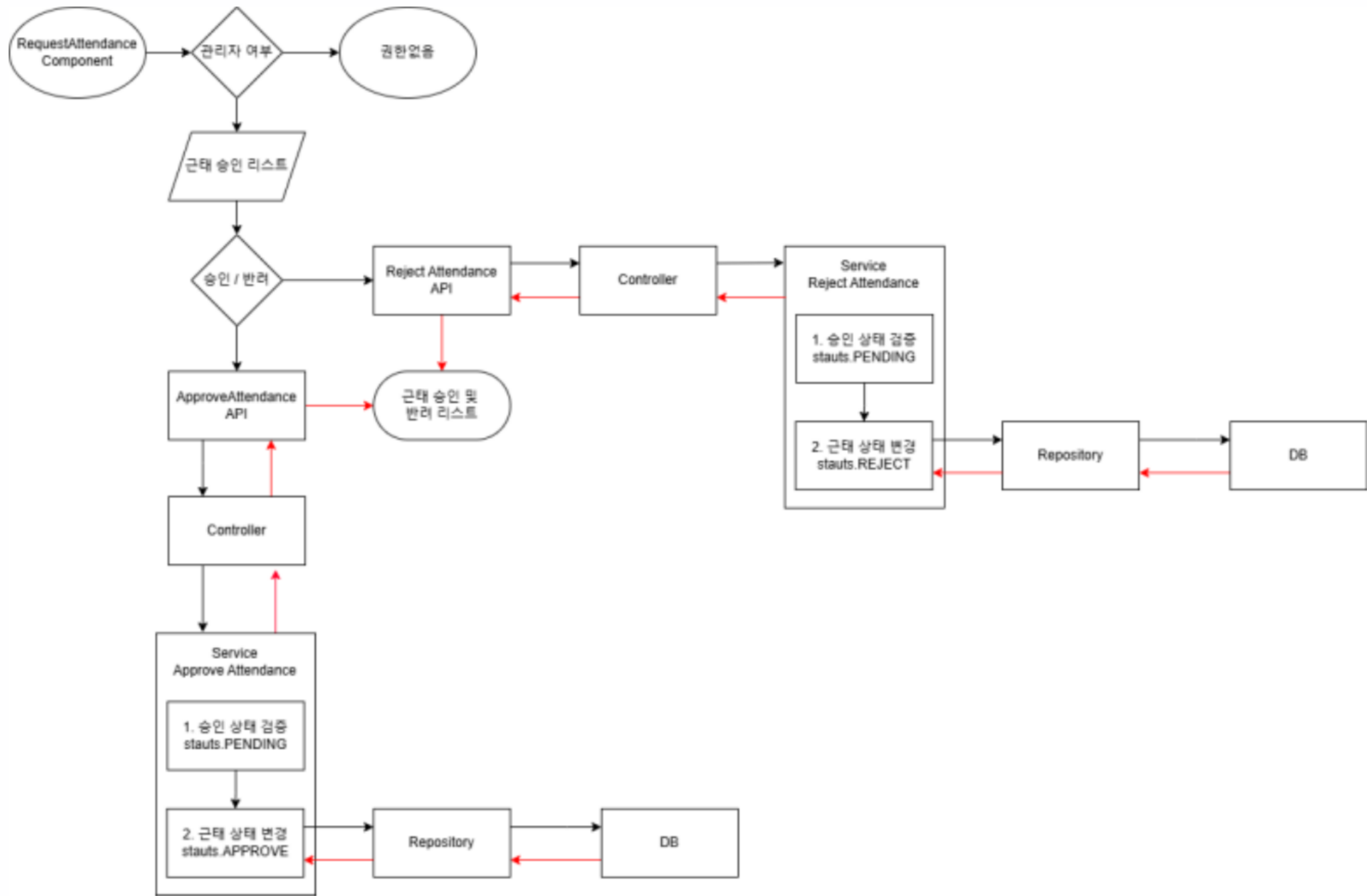
    // 병가는 다음날부터 가능
    if (newStatus == AttendanceStatus.SICK_LEAVE && date.isBefore(today.plusDays( daysToAdd: 1)))) {
        throw new IllegalArgumentException("병가는 다음날부터 신청 가능합니다.");
    }

    // 휴가 & 재택근무는 일주일 뒤부터 가능
    if ((newStatus == AttendanceStatus.LEAVE || newStatus == AttendanceStatus.REMOTE_WORK)
        && date.isBefore(today.plusDays( daysToAdd: 7)))) {
        throw new IllegalArgumentException("휴가 및 재택근무는 일주일 뒤부터 신청 가능합니다.");
    }

    Employee employee = employeeRepository.findById(employeeId)
        .orElseThrow(() -> new IllegalArgumentException("Invalid Employee ID: " + employeeId));
```

- 1, 사용자는 근태 신청 내역에서 "근태 신청" 버튼을 클릭합니다.
- 2. 모달 창에서 근태 유형(병가, 재택, 휴가)과 날짜를 선택하고 사유를 입력합니다.
- 3. 신청 제한 조건(예: 병가는 과거 날짜 신청 불가, 재택근무는 특정 일수 초과 불가)이 적용됩니다.
- 4. 직원 ID를 확인한 후 승인 요청이 진행됩니다.

근태 승인/반려 - 플로우 차트



근태 승인 및 반려 요청

- 1. 관리자 권한이 있는 사람만 근태 승인 페이지 접근 가능
- 2. 승인 및 반려를 하면 마이페이지에서 근태 상태를 확인 가능
- 3. 휴가 및 병가 내역이 승인 상태가 되면 일수 갯수 상태 확인 가능

근태 승인 관리

사원 번호	신청 사원명	신청 유형	신청 날짜	신청 사유	승인 관리
80001234	관리자	병가	2025-03-19	병가	승인반려
30001234	정우성	휴가	2025-03-26	휴가	승인반려
30001234	정우성				

날짜	근태 유형	사유	승인 상태
2025-03-29	휴가	○	반려
2025-03-28	휴가	휴가	승인 대기
2025-03-19	병가	병가	승인

재택근무 내역

날짜	근태 유형
2025-04-05	재택근무

근태 신청

휴가 & 병가 내역

휴가 쓴 일수: 1 / 12

날짜	근태 유형	사유	승인 상태
2025-03-29	휴가	○	반려
2025-03-28	휴가	휴가	승인
2025-03-19	병가	병가	승인

근태 승인 및 반려 요청

```
// 신청 요청
const handleRequest = async () => {
  if (!reason.trim() || !selectedDate) {
    alert("신청 날짜와 사유를 입력해주세요.");
    return;
  }

  if (!employeeId) {
    alert("로그인이 필요합니다.");
    return;
  }

  setLoading(true);
  try {
    await requestApproval(employeeId, selectedDate, reason);
    alert("신청이 완료되었습니다. 승인 대기 중입니다.");
    if (typeof onClose === "function") {
      onClose(); // onClose가 함수일 때만 실행
    }
  } catch (error) {
    alert("신청 실패: " + error.message);
  }
  setLoading(false);
};
```

```
const handleApprove = async (id) => {
  try {
    await approveAttendance(id);
    alert("승인 완료되었습니다.");
    fetchPendingRequests(); // 목록 갱신
  } catch (error) {
    alert("승인 실패: " + error.message);
  }
};
```

```
const handleReject = async (id) => {
  try {
    await rejectAttendance(id);
    alert("거부 완료되었습니다.");
    fetchPendingRequests(); // 목록 갱신
  } catch (error) {
    alert("거부 실패: " + error.message);
  }
};
```

// 승인 거부 처리

@Transactional * paesir

```
public AttendanceDTO rejectAttendance(Integer id) {
    Attendance attendance = attendanceRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("근태 기록을 찾을 수 없습니다: " + id));

    attendance.setRequestStatus(RequestStatus.REJECTED);
    return convertToDTO(attendanceRepository.save(attendance));
}
```

@Transactional * paesir *

```
public AttendanceDTO approveAttendance(Integer id) {
    Attendance attendance = attendanceRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("근태 기록을 찾을 수 없습니다: " + id));

    attendance.setRequestStatus(RequestStatus.APPROVED); // 승인 상태로 변경

    return convertToDTO(attendanceRepository.save(attendance));
}
```

1. 관리자는 근태 신청을 조회하고 승인 또는 반려를 선택할 수 있습니다.

2. 승인 시 APPROVED, 반려 시 REJECTED 상태로 변경됩니다.

3. 선택한 상태와 사유를 서버로 전송하여 요청을 처리합니다.

4. 처리 결과에 따라 성공 또는 오류 메시지가 표시됩니다.

근태 내역 - 마이페이지

My 출/퇴근 조회

직원 ID: 80001234

출근

퇴근

당일 출퇴근 내역

날짜	직원번호	출근 시간	퇴근 시간	상태	초과 근무(시간)
2025-03-16	80001234	17:58:51	퇴근 전	지각	0

전체 출퇴근 내역

날짜	직원번호	출근 시간	퇴근 시간	상태	초과 근무(시간)
2025-04-05	80001234	출근 전	퇴근 전		
2025-03-29	80001234				
2025-03-28	80001234				
2025-03-19	80001234				
2025-03-14	80001234				

```
// 출근 기록이 없는 경우 새로 생성
LocalTime checkInTime = dto.getCheckInTime() != null ? dto.getCheckInTime() : LocalTime.now();
AttendanceStatus status = checkInTime.isBefore(LocalTime.of( hour: 9, minute: 0)) ? AttendanceStatus.PRESENT : AttendanceStatus.LATE;

Attendance attendance = Attendance.builder()
    .employee(employee)
    .date(dto.getDate())
    .checkInTime(checkInTime) // 현재 시간 설정
    .checkOutTime(null) // 출근 시점이므로 퇴근 시간 없음
    .status(status) // 상태 값 저장 (출근 상태 유지)
    .requestStatus(RequestStatus.APPROVED) // 기본값 적용
    .overtimeHours(BigDecimal.ZERO) // 기본값
    .overtimeFormatted("0시간 0분 0초") // 초 단위 추가
    .build();

return convertToDTO(attendanceRepository.save(attendance));
}
```

```
// | 퇴근 기록 업데이트 (퇴근 버튼 클릭)
Attendance attendance = existingAttendance.get();

if (dto.getCheckOutTime() != null) {
    attendance.setCheckOutTime(dto.getCheckOutTime());

    // 출근 시간과 퇴근 시간이 모두 존재해야 초과 근무 계산 가능
    if (attendance.getCheckInTime() != null) {
        Duration totalWorkDuration = Duration.between(attendance.getCheckInTime(), attendance.getCheckOutTime());
        long totalWorkSeconds = totalWorkDuration.getSeconds(); // 총 근무 시간(초 단위)
        long totalWorkHours = totalWorkSeconds / 3600; // 총 근무 시간(시간 단위)

        if (totalWorkHours > 10) { // 10시간 초과 근무일 경우만 계산
            long overtimeSeconds = totalWorkSeconds - (10 * 3600); // 초과 근무 시간 (10시간을 뺀 나머지)
            long overtimeHours = overtimeSeconds / 3600;
            long overtimeMinutes = (overtimeSeconds % 3600) / 60;
            long overtimeSecs = overtimeSeconds % 60;
```


직원 자동 퇴사 처리

직원 관리

사원명

검색어를 입력하세요.

검색

직원 등록

사원 번호	이름	생년월일	성별	전화번호	이메일	주소	부서	직급	입사일	퇴사 여부
10000001	영업팀 사원	1990-01-01	남성	010-0000-0000	useongj490@gmail.com	서울 강남	영업팀	사원	2025-02-22	● 재직 중
10000002	영업팀 대리	1990-01-01	여성	010-0000-0000	useongj490@gmail.com	서울 강남	영업팀	대리	2025-02-22	● 재직 중
10000003	영업팀 부장	1990-01-01	여성	010-0000-0000	useongj490@gmail.com	서울 강남	영업팀	부장	2025-02-22	● 재직 중
10001234	황수림	1990-01-01	남성	010-0000-0000	akdlgosvhs@gmail.com	서울 강남	영업팀	이사	2025-02-22	● 재직 중
10007777	박남수	1990-01-01	여성	010-0000-0000	useongj490@gmail.com	서울 강남	영업팀	이사	2025-02-22	● 재직 중
20000001	회계팀 인턴	1990-01-01	남성	010-0000-0000	useongj490@gmail.com	서울 강남	회계팀	인턴	2025-02-22	● 재직 중
20000002	회계팀 과장	1990-01-01	여성	010-0000-0000	useongj490@gmail.com	서울 강남	회계팀	과장	2025-02-22	● 재직 중
20001234	이정현	1990-01-01	여성	010-7491-7490	jay6423@gmail.com	서울 강남	회계팀	이사	2025-02-22	● 재직 중
30000003	인사팀 부장	1990-01-01	여성	010-0000-0000	useongj490					
30001234	정우성	1990-01-01	여성	010-0000-0000	useongj490					

직원 상세 정보

X

사원 번호	30001234	이름	정우성
생년월일	1990-01-01	전화번호	010-0000-0000
이메일	useongj490@gmail.com	주소	서울 강남
부서	인사팀	직급	이사
입사일	2025-02-22	퇴사일	2025-03-15

수정 요청

1. 직원 관리 페이지에서 Pagination 데이터 확인

2. 직원 이름을 클릭 하여 모달을 통해 세부 정보 확인 후 퇴사일 지정 후 수정 요청

3. 승인이 완료가 되면 해당 되는 날 자동 퇴사 처리

직원 자동 퇴사 처리

```
/** 매일 자정에 실행되어 퇴사일이 지난 직원들을 퇴사 처리 */
@Scheduled(cron = "0 18 12 * * ?") // 매일 11:51 실행  🧑 paesir *   매 12:18
@Transactional
public void processEmployeeTerminations() {
    LocalDate today = LocalDate.now();
    System.out.println("🕒 [퇴사 스케줄러] 실행됨: " + today);

    // 퇴사일이 오늘 이전이거나, 오늘 날짜이면서 퇴사되지 않은 직원 찾기
    List<Employee> employeesToTerminate = employeeRepository.findByTerminationDateBeforeOrTerminationDateEqualsAndIsTerminatedFalse(todayBeforeOrToday);

    // `is_terminated = true` 이면서 `PENDING` 상태인 직원도 자동 퇴사 처리 대상에 포함
    List<Employee> pendingEmployeesToTerminate = employeeRepository.findByTerminationDateBeforeOrTerminationDateEqualsAndIsTerminatedTrueAndStatus(todayBeforeOrToday, "PENDING");

    // 퇴사 자동 처리
    List<Employee> findByTerminationDateBeforeOrTerminationDateEqualsAndIsTerminatedFalse(LocalDate todayBeforeOrToday) {
        return employeeRepository.findAll();
    }

    // is_terminated = true 이면서 PENDING 상태이고, 퇴사일이 오늘이거나 지난 직원 조회
    List<Employee> findByTerminationDateBeforeOrTerminationDateEqualsAndIsTerminatedTrueAndStatus(LocalDate todayBeforeOrToday, String status) {
        return employeeRepository.findAll();
    }
}
```

해당 날짜까지 관리자의 승인 여부와 관계없이, 시스템은 스케줄러를 통해 상태값을 조회하고, 지정된 시간에 자동으로 퇴사 처리를 수행합니다. 이를 통해 관리자의 수동 개입 없이도 정해진 일정에 따라 퇴사 절차가 자동화됩니다.

🕒 [회사 스케줄러] 일반 퇴사 대상 직원 수: 0

🕒 [회사 스케줄러] PENDING 상태에서 이미 퇴사 처리된 직원 수: 1

✳️ [회사 스케줄러] 처리 중인 직원: ID=80001239, 이름=영업팀 부장, 퇴사일=2025-03-14, 현재 상태=PENDING, 퇴사 처리 상태=true

✅ [자동 승인] 직원 ID=80001239 - 영업팀 부장

✅ [회사 처리 완료] 직원 ID=80001239 - 영업팀 부장

Hibernate:
delete
from
member
where
employee_id=?

📌 [회사 스케줄러] 직원 계정 삭제 완료: ID=80001239

✅ [회사 스케줄러] 모든 직원 퇴사 처리 완료!

Hibernate:
update
employee
set
address=?,
approved_by_employee_id=?,
birth_date=?,
department_id=?,
email=?,
gender=?,
hire_date=?,
is_terminated=?,
name=?,
parent_employee_id=?,
phone=?

페이지네이션

주문 내역

날짜 선택: 연도: 월: 일: ~ 연도: 월: 일: 검색

	거래 ID	구분	거래처명	거래일자	제품명	주문 수량	주문 금액	주문 담당자
☐	51	판매	테크플러스	2024-05-08	Apple MacBook Pro 14	12	35,640,000원	박남수
☐	52	판매	테크플러스	2024-09-13	Panasonic KX-TGF382M	1	275,000원	황수원
☐	53	판매	ABC Supply Co.	2024-06-02	Logitech MX Keys	8	1,584,000원	박남수
☐	54	판매	XYZ Materials Ltd.	2024-07-09	Synology DS920+	18	15,840,000원	박남수
☐	55	판매	하이엔드PC	2024-11-19	APC Smart-UPS 1500VA	1	770,000원	박남수
☐	56	판매	테크플러스	2024-01-21	Epson EB-1781W	20	30,800,000원	황수원
☐	57	판매	PC마켓	2024-08-01	Microsoft Surface Pro 9	2	7,700,000원	박남수
☐	58	판매	하이엔드PC	2024-10-03	HP LaserJet Pro M404dn	4	1,980,000원	황수원
☐	59	판매	피씨허브	2024-08-25	Cisco Business 250 Series	4	2,200,000원	황수원
☐	60	판매	피씨허브	2024-12-22	Microsoft Surface Pro 9	2	3,850,000원	황수원

<< 1 2 3 4 5 6 7 >>

```
    )}
  </tbody>
</table>
</div>

{totalPages > 1 && (
  <Pagination
    currentPage={currentPage}
    totalPages={totalPages}
    onPageChange={onPageChange}
  />
)}
</div>
);
};
```

🔬 🌱

페이지네이션 처리 FE vs BE ?

- 1. 백엔드에서 전체 페이지네이션 처리를 끝내는 방식
- 2. 백엔드에서 Page값만 반환시키고 최종 처리는 프론트엔드에서 하는 방식

what?

이 프로젝트에서는 동일 포맷의 css가 반복되기 때문에 재사용성을 위해 프론트엔드에서 만들어 버튼 css까지 동일하게 사용하기 위해 프론트엔드에서 처리하기로 생각하고 프론트엔드에서 처리

how?

Pagination 컴포넌트

```
/* Pagination 컴포넌트 사용법
1. 해당 부모 페이지에 useState 셋업
const [page, setPage] = useState(0);
const [size] = useState(10);
const [totalPages, setTotalPages] = useState(1);

2. async 보낼때 page 포함해서 보내야함
예시 :
const fetchClients = async (page) => {
  try {
    const data = await getAllClients(page, size);
    setClients(data.content);
    setTotalPages(data.totalPages);
  } catch (error) {
    console.error("Error fetching clients:", error);
  }
};

3. return 구현하는 곳에
{!searchResults && totalPages > 1 && (
  <Pagination
    currentPage={page}
    totalPages={totalPages}
    onPageChange={setPage}
  />
)}
// !searchResults && totalPaes > 1 이 부분에서 searchResults 는 상황에 따라 다르게 설정할 수 있음
*/
```


검색어 자동 완성

기업명 ▼

한국

검색

한국 HP 주식회사

한국마이크로소프트

입력



1. 검색어 자동완성을 위해 Repository 쿼리문 작성

```
public interface ClientRepository extends JpaRepository<Client, Long> {  
  
    @Query("SELECT c FROM Client c WHERE c.clientName LIKE %:clientName%")  
    Page<Client> findByClientName(String clientName, Pageable pageable);  
  
}
```



2. 상태관리를 위한 useState 추가

```
const [suggestions, setSuggestions] = useState([]);  
const [selectedIndex, setSelectedIndex] = useState(0);  
const [isSelecting, setIsSelecting] = useState(false);
```



3. 이벤트 핸들러 및 useEffect 설정

```
// 검색어 변경 시 자동완성 실행  
useEffect(() => {  
    if (searchParams.searchTerm.length > 1 && !isSelecting) {  
        fetchSuggestions(searchParams.searchTerm, searchParams.searchType);  
    } else {  
        setSuggestions([]);  
    }  
    setIsSelecting(false);  
}, [searchParams.searchTerm, searchParams.searchType]);  
  
// 입력값 변경 핸들러  
const handleInputChange = (e) => {  
    setSearchParams(prev => ({ ...prev, [e.target.name]: e.target.value }));  
};  
  
// 키보드 이벤트 핸들러  
const handleKeyDown = (e) => {  
    if (suggestions.length === 0) return;  
  
    if (e.key === 'ArrowDown') {  
        setSelectedIndex(prev => (prev < suggestions.length - 1 ? prev + 1 : prev));  
    } else if (e.key === 'ArrowUp') {  
        setSelectedIndex(prev => (prev > 0 ? prev - 1 : prev));  
    } else if (e.key === 'Enter') {  
        if (selectedIndex < 0 || selectedIndex >= suggestions.length) return; // 인덱스 범위 체크  
  
        const selectedItem = suggestions[selectedIndex];  
        if (!selectedItem) return; // 선택된 항목이 존재하는지 확인  
  
        const selectedValue = selectedItem?.clientCode || selectedItem?.clientName;  
        if (!selectedValue) return; // 값이 없는 경우 리턴  
  
        setIsSelecting(true);  
        setSearchParams(prev => ({ ...prev, searchTerm: selectedValue }));  
        setSuggestions([]);  
        e.preventDefault();  
    }  
};
```

6. 문제 해결 - 웹소켓

```
@socketio.on("message")
def handle_message(data):
    chat_room_id = data.get("chatRoomId")
    sender_id = data.get("senderId")
    receiver_id = data.get("receiverId")
    message_text = data.get("messageText")

    print(f" [Flask] 수신 데이터: {json.dumps(data, indent=2, ensure_ascii=False)}")

    if not message_text:
        print(" 메시지가 비어 있습니다.")
        return

    # WebSocket으로 보낼 메시지
    websocket_message = {
        "chatRoomId": chat_room_id,
        "senderId": sender_id,
        "receiverId": receiver_id,
        "messageText": message_text,
        "content": message_text, # React에서 사용
        "timestamp": datetime.datetime.now().isoformat(), # 현재 시간 추가
    }

    # 수신자(receiver_id)에게만 메시지 전송 (보낸 사람에게는 안 보냄)
    if receiver_id in connected_users:
        for sid in connected_users[receiver_id]:
            print(f" WebSocket 메시지 전송 → 수신자 {receiver_id}, SID: {sid}")
            socketio.emit("message", websocket_message, room=sid)
            socketio.sleep(0) # WebSocket 이벤트 강제 실행
        print(f" WebSocket 메시지 전송 완료 → 수신자 {receiver_id}")
    else:
        print(f" 수신자 {receiver_id}가 WebSocket에 연결되지 않음.")
```

보완 전

WebSocket을 통해 실시간으로 메시지를 받아왔지만
메신저의 WebSocket이 끊어지면 메시지를 가져올 방법이 없고
사용자가 오프라인일 때 메시지를 확인할 수 없는 문제가 발생



```
const lastMessageMap = {};
for (const room of filteredRooms) {
    const messages = await getMessages(room.id);
    lastMessageMap[room.id] =
        messages.length > 0
        ? {
            content: messages[messages.length - 1].content,
            timestamp: messages[messages.length - 1].timestamp,
        }
        : { content: "대화 없음", timestamp: "" };
    setLastMessages(lastMessageMap);
} catch (error) {
    console.error("X 데이터 불러오기 오류:", error);
}
};
```

보완 후

WebSocket이 끊어지면 RESTful API(getMessages, getUnreadMessages)를 사용하여 메시지를 가져오도록 개선
더 이상 WebSocket이 연결되지 않아도 사용자가 채팅 내용을 확인 가능

7. 느낀점

구분	내용
어려움 극복	Python, React, Spring Boot를 합쳐서 풀스택으로 챗봇을 구현하는 방법이 어려웠지만 배운 기본적인 개념을 바탕으로 자연어 처리와 데이터를 연동하여 실시간으로 백단의 데이터를 조회하여 응답할 수 있도록 AI 챗봇을 구현할 수 있었습니다.
잘한 점	초반 뷰 페이지의 기능 정리를 디테일하게 구상한 덕분에 큰 어려움 없이 구현을 할 수 있었습니다. Python과 Socket.IO을 활용하여 구현한 실시간 사내 메신저와 알림, AI 챗봇을 통해 사용자에게 업무의 효율성과 편의성이 향상했습니다. 모델링을 탄탄하게 설계하여 각 테이블간의 관계를 유연하게 설정했습니다. 이를 통해 데이터 처리와 기능 구현이 원활하게 진행되었습니다.
아쉬운 점	ERP를 구현하면서 실제 비즈니스 프로세스를 반영해서 하다보니 각 부서별로 어떤 데이터들이 필요하고 활용해야할지 파악하기 어려웠습니다. 때문에 팀원들끼리 함께 협력하며 부서별 업무사항을 분석하여 시스템 설계를 탄탄히 구현하려고 노력했습니다. 초반에는 모든 기능을 한 번에 처리하다려다보니 코드가 복잡해지고, 꼬이면서 코드를 다시 구현해야하는 일이 발생했습니다. 그래서 기능별로 우선순위를 명확하게 정하고 코드의 리팩토링을 통해 중복된 부분을 처리하고, 가독성과 유지보수성을 높이기 위해 코드 구조를 개선했습니다.

THANK YOU
